29.4 ClawHub 技能注册中心

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~430k tokens，输出 ~5k tokens（本节）

内置技能和工作区技能满足了大多数场景，但技能系统的真正威力在于**社区生态**。ClawHub 是 OpenClaw 的技能注册中心，让开发者可以发布、搜索和安装第三方技能——类似于 npm 之于 Node.js，或 Homebrew 之于 macOS。

29.4.1 ClawHub 的设计与功能

注册中心架构

ClawHub（clawhub.com）是一个中心化的技能托管服务，提供以下核心功能：

功能	说明
搜索	基于关键词搜索技能（如 "postgres backup"）
安装	将技能下载到本地 `~/.openclaw/skills/` 目录
版本管理	支持语义化版本，可指定安装特定版本
更新	基于哈希比对的增量更新，支持批量更新
发布	开发者上传技能到注册中心
认证	`clawhub login` 进行身份验证（发布时需要）

ClawHub CLI

ClawHub 通过独立的 CLI 工具交互，本身也是一个 OpenClaw 技能（自举模式）：

```
# 安装 ClawHub CLI
npm i -g clawhub

# 认证（发布需要）
clawhub login
clawhub whoami

# 搜索技能
clawhub search "postgres backups"

# 安装技能
clawhub install my-skill
clawhub install my-skill --version 1.2.3

# 更新技能
clawhub update my-skill
clawhub update --all                    # 更新所有已安装技能
clawhub update my-skill --force         # 强制更新（忽略哈希匹配）

# 列出已安装技能
clawhub list

# 发布技能
clawhub publish ./my-skill \
  --slug my-skill \
  --name "My Skill" \
  --version 1.2.0 \
  --changelog "Fixes + docs"
```

安装目录

ClawHub 安装的技能存放在 `~/.openclaw/skills/` 目录（即 Managed Skills 目录）：

```
~/.openclaw/skills/
├── postgres-backup/
│   └── SKILL.md
├── linear-issues/
│   └── SKILL.md
└── custom-tool/
    ├── SKILL.md
    └── helper.sh
```

这些技能的优先级高于 Bundled（内置），但低于 Workspace（项目级）。

更新机制

ClawHub 的更新采用**基于哈希的版本比对**：

1. 计算本地技能文件的哈希值
2. 发送到注册中心进行匹配
3. 如果哈希匹配某个已知版本，升级到最新版本
4. 如果哈希不匹配（本地已被修改），跳过更新（除非 `--force`）

这个设计在自动更新和本地修改之间取得了平衡——如果用户手动修改了安装的技能文件，`clawhub update` 不会静默覆盖这些修改。

注册中心配置

默认注册中心是 `https://clawhub.com`，可通过环境变量或参数覆盖：

```
# 使用私有注册中心
CLAWHUB_REGISTRY=https://internal.example.com clawhub search "tools"

# 或通过参数
clawhub install my-skill --registry https://internal.example.com
```

工作目录默认为当前目录（回退到 OpenClaw 工作区），安装目录默认为 `./skills`：

```
# 自定义工作目录
CLAWHUB_WORKDIR=/path/to/workspace clawhub install my-skill
clawhub install my-skill --workdir /path/to/workspace
```

29.4.2 技能自动搜索与安装

Agent 驱动的技能发现

ClawHub 最强大的用法不是手动操作——而是让 Agent 自主搜索和安装技能。由于 `clawhub` 本身是一个技能，Agent 知道如何使用它：

场景：用户说 "帮我查看 Linear 上的待办事项"

1. Agent 检查已激活技能列表，发现没有 Linear 相关技能
2. Agent 执行 `clawhub search "linear"`
3. 搜索结果返回 `linear-issues` 技能
4. Agent 执行 `clawhub install linear-issues`
5. 技能安装到 `~/.openclaw/skills/linear-issues/`
6. 下次会话启动时， `linear-issues` 技能自动激活
7. Agent 根据新技能中的指引操作 Linear API

这实现了一种**运行时能力扩展**——Agent 可以在对话过程中获得新能力，而不需要重启或手动配置。

安全考量

第三方技能带来了安全风险——恶意技能可能包含有害指令。OpenClaw 通过多层防护应对：

1. **安装前扫描**： `scanDirectoryWithSummary` 检查技能目录中的可疑代码模式
2. **权限系统**：技能内容注入提示词后，Agent 的实际操作仍受限于工具权限（如 bash 的命令白名单）
3. **哈希验证**：ClawHub 记录每个版本的内容哈希，防止篡改
4. **审计命令**： `openclaw security audit --deep` 可深度扫描所有已安装技能

技能发布流程

开发者创建新技能并发布到 ClawHub 的典型流程：

```
# 1. 创建技能目录
mkdir my-awesome-skill && cd my-awesome-skill

# 2. 编写 SKILL.md（也可以用 skill-creator 技能辅助）
cat > SKILL.md << 'EOF'
---
name: my-awesome-skill
description: Do amazing things with the Awesome API.
metadata:
  openclaw:
    requires: { env: ["AWESOME_API_KEY"] }
---

# My Awesome Skill

## Usage
...
EOF

# 3. 测试（放到项目的 skills/ 目录中验证）
cp -r . ~/my-project/skills/my-awesome-skill/

# 4. 发布
clawhub login
clawhub publish . --slug my-awesome-skill --name "My Awesome Skill" --version 1.0.0
```

OpenClaw 还内置了 `skill-creator` 技能，可以在对话中引导用户创建新技能——Agent 会询问技能名称、描述、依赖，然后生成标准的 `SKILL.md` 文件。

本节小结

1. **ClawHub** 是 OpenClaw 的技能注册中心，提供搜索、安装、版本管理、更新和发布功能，类似于 npm 生态。
2. **基于哈希的更新**在自动更新和本地修改保护之间取得平衡——修改过的技能不会被静默覆盖。
3. **Agent 驱动的技能发现**让 Agent 能在对话中自主搜索和安装新技能，实现运行时能力扩展。
4. **多层安全防护**覆盖安装前扫描、工具权限控制、哈希验证和深度审计，防范恶意第三方技能。
5. **自举生态**：`clawhub` 和 `skill-creator` 都是技能，技能系统用自身来管理和创建技能，形成良性循环。